

Friend Functions/Classes & Memory Management

Note: Both halves of this topic are C++-specific — friend functions/classes don't exist in Java at all (Java has no equivalent access-granting mechanism), and manual `new` / `delete` memory management is exactly what Java's garbage collector exists to eliminate. Java's contrast is noted throughout.

PART 1 — Friend Functions & Friend Classes

1. The Problem: Encapsulation Is Sometimes *Too* Strict

Encapsulation (recall OOP Fundamentals) hides a class's private data from everything outside it — a good default. But occasionally, a **specific**, trusted external function or class genuinely needs direct access to another class's private internals, without making that data broadly public (which would remove protection from *everyone*, not just the one trusted party).

friend is C++'s controlled escape hatch: it grants **one specific** external function or class direct access to a class's private/protected members, **without** making that data public to everyone else.

2. Friend Functions

```
class EquilateralTriangle {
    int side;
    int area, circumference;
public:
    EquilateralTriangle(int s) : side(s) {
        area = 1.732 * side * side / 2;
        circumference = 3 * side;
    }
    friend void print(EquilateralTriangle); // grants THIS function access
};

void print(EquilateralTriangle et) {
    cout << "Area: " << et.area << endl // accessing PRIVATE members directly
    << "Circumference: " << et.circumference << endl;
}
```

- `print()` is declared **friend** **inside** the class it needs access to — the class itself decides who to trust; a function cannot unilaterally declare itself a friend of some other class.

- Once declared, `print()` can read `et.area` and `et.circumference` directly, even though they're `private` — despite `print()` **not** being a member function of `EquilateralTriangle` at all.

3. Friend Classes

```
class EquilateralTriangle {
    int side;
public:
    EquilateralTriangle(int s) : side(s) {}
    friend class Calculator;    // grants an ENTIRE class access
};
class Calculator {
public:
    int computeArea(EquilateralTriangle& et) {
        return 1.732 * et.side * et.side / 2;    // Calculator can access ALL private members
    }
};
```

`friend class Calculator;` grants **every** member function of `Calculator` direct access to `EquilateralTriangle`'s private members — a broader grant than a single friend function.

4. Properties of Friendship — the rules interviewers actually probe

Property	Rule
Not mutual	<code>A</code> declaring <code>B</code> as a friend does not automatically make <code>A</code> a friend of <code>B</code> — friendship must be granted explicitly, in each direction, separately
Not inherited	If <code>Base</code> grants friendship to <code>F</code> , a class <code>Derived : public Base</code> does not automatically extend that friendship — <code>F</code> still cannot access <code>Derived</code> 's own new private members (though it retains access to the inherited <code>Base</code> portion, since that access was already granted at the <code>Base</code> level)
Not transitive	If <code>A</code> is a friend of <code>B</code> , and <code>B</code> is a friend of <code>C</code> , <code>A</code> is not automatically a friend of <code>C</code> — each friendship grant is a single, direct, non-chaining relationship

Interview soundbite: "Friendship in C++ is a *deliberate, one-way, non-chaining* grant of trust — every single one of those three properties (not mutual, not inherited, not transitive) exists to keep the encapsulation escape hatch as narrow and explicit as possible. If any of these were automatic, `friend` would spread trust unpredictably through a codebase, defeating the whole purpose of it being a *targeted* exception rather than a general weakening of encapsulation."

5. When (and When Not) to Use `friend`

Legitimate uses:

- Operator overloading where the left operand can't be the class itself (recall `operator<<` from the Operator Overloading topic — this is the single most common, well-justified use of `friend` in real code).
- Two tightly-coupled classes that are conceptually part of the same abstraction (e.g., a linked list and its own iterator class) needing to share internals without exposing them publicly.

Overuse is a design smell: reaching for `friend` broadly, between classes that aren't tightly conceptually coupled, is usually a sign that the class boundaries themselves are drawn in the wrong place — it's frequently cleaner to add a well-designed public method that provides exactly the needed access, rather than granting blanket private access via `friend`.

Java has no `friend` mechanism at all. The closest equivalent is Java's **package-private** (default) access — members with no access modifier are accessible to any class in the same package, which is a broader, less targeted form of controlled visibility than C++'s one-to-one friend grants (developed further in the Access Modifiers topic).

PART 2 — Memory Management: `new` and `delete`

6. `new` and `delete` — the basics

```
int* ptr = new int(5);      // allocate a SINGLE int on the heap, initialized to 5
int* arr = new int[3];      // allocate an ARRAY of 3 ints on the heap
for (int i = 0; i < 3; i++) { arr[i] = i; cout << arr[i] << endl; }

delete ptr;                // free the single int
delete[] arr;              // free the ARRAY — note the []
```

- `new` — allocates memory on the **heap**, **calls the constructor** (for object types), and returns a **typed pointer** to the allocated memory.
- `delete` — calls the **destructor** (for object types), then frees the memory back to the system.

The `new` / `delete` vs. `new[]` / `delete[]` matching rule — a classic, dangerous bug

`new` must be paired with `delete` ; `new[]` must be paired with `delete[]` . Mixing them is undefined behavior.

```
int* arr = new int[10];
delete arr;      // WRONG – should be delete[] arr;
                // For objects (not just plain ints), this specifically only calls the
                // destructor for the FIRST element, leaving the other 9 objects'
                // destructors never called – a partial resource leak, plus technically UB.
```

Why this distinction exists: `new[]` stores extra bookkeeping (typically the array's length) alongside the allocated block, so `delete[]` knows how many destructors to call before freeing the memory. Plain `delete` doesn't know to look for or use that bookkeeping — it assumes a single object, and calls exactly one destructor.

new vs. C's malloc

	new	malloc
Calls constructor?	Yes	No — just raw memory
Type-safe?	Yes — returns a typed pointer	No — returns <code>void*</code> , requires an explicit cast
Corresponding deallocation	delete	free
Can be overloaded?	Yes (per-class custom allocation)	No

7. The Three Classic Memory Bugs

Bug	What happens	Cause
Memory leak	Allocated memory is never freed	Forgetting <code>delete</code> / <code>delete[]</code> , or losing the only pointer to it before freeing
Dangling pointer	A pointer still refers to memory that's already been freed	Using a pointer <i>after</i> <code>delete</code> , without setting it to <code>nullptr</code>
Double free	<code>delete</code> is called twice on the same memory	Often from a shallow copy (recall Constructors & Destructors) where two objects' destructors both try to free the same pointer

Guarding against dangling pointers — a simple habit:

```
delete ptr;
ptr = nullptr; // now any accidental later use of ptr crashes IMMEDIATELY and OBVIOUSLY,
               // rather than silently corrupting memory it no longer legitimately owns
```

Why setting to `nullptr` after `delete` is good practice, not just paranoia: using a dangling (non-null) pointer after `delete` is undefined behavior that might *appear* to work for a while (the memory often isn't immediately overwritten), making the bug extremely hard to track down later, far from its actual cause. Dereferencing a `nullptr`, by contrast, reliably crashes **immediately**, at the exact point of misuse — turning a silent, delayed corruption bug into an obvious, immediate, easy-to-debug one.

8. The Modern C++ Fix: Smart Pointers

Manual `new` / `delete` pairing is genuinely error-prone (as every bug above demonstrates) — modern C++ strongly favors **smart pointers**, which automatically manage the `delete` call via RAII (Resource Acquisition Is Initialization — tying a resource's lifetime to an object's scope):

Smart pointer	Ownership model	Behavior
<code>unique_ptr<T></code>	Exactly one owner at a time	Automatically deletes when it goes out of scope; cannot be copied (only <i>moved</i>), preventing double-free by construction
<code>shared_ptr<T></code>	Shared ownership, reference-counted	Deletes only when the last owning <code>shared_ptr</code> is destroyed
<code>weak_ptr<T></code>	A non-owning observer of a <code>shared_ptr</code>	Doesn't keep the object alive by itself; used to break reference cycles between <code>shared_ptr</code> s

```
{
    unique_ptr<Buffer> buf = make_unique<Buffer>(10);
    // ... use buf ...
} // buf's destructor runs automatically here — no explicit delete needed, and it can't leak
```

Interview soundbite: "Smart pointers don't add a new capability — they encode the exact 'who owns this, and when should it be freed' discipline a careful C++ programmer would apply manually, but make the compiler enforce it, so a forgotten `delete` or an accidental double-free becomes structurally impossible rather than just a matter of writing careful, disciplined code."

9. Java's Contrast: Garbage Collection

Java has **no** `new` / `delete` symmetry at all — `new` allocates, but there is no `delete`. The **garbage collector** automatically identifies objects with no remaining reachable references and reclaims their memory, at a time of its own choosing (not deterministically controlled by the program).

-  Eliminates memory leaks (in the classic C++ sense), dangling pointers, and double-frees entirely, by construction.
 -  Introduces a **different** class of problem: a **logical memory leak**, where an object is technically still *reachable* (some reference to it still exists somewhere, e.g., in a collection that's never cleared) even though the program logically no longer needs it — the garbage collector can never reclaim it, since reachability, not actual need, is all it can observe.
-

Interview Questions With Answers

Q1. Why is `friend` described as a "controlled escape hatch" rather than simply a hole in encapsulation?

Answer: Unlike making a member `public` (which grants access to literally everything, forever), `friend` grants access to one specific, explicitly named function or class, decided by the class itself in its own definition — the class retains full control over exactly who gets this exception and exactly what they get access to, while everything and everyone else remains fully encapsulated as before. It's "controlled" precisely because it's a targeted, class-authored exception rather than a general weakening of the encapsulation boundary for all callers.

Q2. Why is friendship in C++ neither mutual, inherited, nor transitive? What would go wrong if any of these were automatic?

Answer: Each property exists to keep `friend` narrowly scoped and predictable. If friendship were mutual, granting access in one direction would silently also grant it in reverse — an unintended, un-reviewed exposure. If it were inherited, a class could unknowingly expose its own new private members to a friend that was only ever granted access to some unrelated ancestor class, without ever being asked. If it were transitive, granting friendship to one class could silently cascade trust through a chain of unrelated classes that class happens to also trust, with no direct relationship or review at all. Making none of these automatic ensures every actual grant of access is explicit, deliberate, and reviewable — friendship never spreads further than exactly what was written.

Q3. Why must `operator<<` for custom output typically be declared as a friend function rather than a member function? (Connecting back to Operator Overloading.)

Answer: A member function's implicit left operand is always the object it's called on (`*this`) — but `cout << myObject` has `cout` (an `ostream`) as the left operand, not the custom class, so there's no way to define this as a member function of the custom class at all. It must be a free function taking both operands explicitly, and it's declared `friend` specifically so that free function can still access the custom class's private members directly, without needing public getters just to support printing.

Q4. What specifically goes wrong if you allocate an array with `new[]` but deallocate it with plain `delete` instead of `delete[]` ?

Answer: `new[]` allocates the array along with hidden bookkeeping (typically the element count) that `delete[]` uses to know how many destructors to call before freeing the underlying memory block. Plain `delete` has no awareness of this bookkeeping and assumes it's freeing a single object — for a class type, it will call the destructor for only the *first* element in the array, silently skipping destructor calls for every other element (a partial resource leak for whatever those other elements' destructors were supposed to clean up), and the deallocation itself is technically undefined behavior, since the memory wasn't allocated in the form plain `delete` expects to free.

Q5. Why does setting a pointer to `nullptr` immediately after calling `delete` on it help catch bugs, even though it doesn't prevent the underlying issue (using a pointer after its memory is freed)?

Answer: The underlying issue — some code path later trying to use a pointer whose memory has already been freed — isn't actually prevented by setting it to `nullptr` ; if that later code runs, it will still be wrong. What changes is the *failure mode*: dereferencing a dangling (non-null but freed) pointer is undefined behavior that might silently "work" for a while, since the memory often isn't immediately reused or overwritten — making the bug's actual root cause extremely hard to trace later, far from where it was actually introduced. Dereferencing a `nullptr` instead crashes immediately and obviously, at the exact point of the erroneous later use — converting a silent, delayed-manifesting bug into an immediate, loud, easy-to-locate one.

Q6. How does `unique_ptr` prevent double-free bugs by construction, rather than just by convention/discipline?

Answer: `unique_ptr` is specifically designed to be **non-copyable** — attempting to copy a `unique_ptr` (which would create two owning pointers to the same memory, exactly the setup that leads to a double-free when both eventually get destroyed) is a compile-time error, not just a bad practice that a careful programmer avoids. It can only be *moved* (transferring ownership, leaving the source `unique_ptr` empty/null) — at any given moment, there is structurally, provably, only ever one `unique_ptr` that considers itself the owner of a given piece of memory, so exactly one destructor call will ever attempt to free it, eliminating the double-free scenario as a class of bug entirely, enforced by the type system rather than relying on programmer discipline.

Q7. Why doesn't Java's garbage collection eliminate memory leaks entirely, even though it removes the classic C++ "forgot to call delete" problem?

Answer: Java's garbage collector reclaims memory based purely on **reachability** — whether some chain of references still leads to an object — not based on whether the program actually still *needs* that object. It's entirely possible for an object to remain reachable (e.g., still referenced by an entry in a cache or collection that's simply never cleared or removed from) long after the program has logically finished using it — the garbage collector has no way to know the program doesn't need it anymore, since reachability says otherwise, so it never reclaims that memory. This is a genuinely different class of bug

(a "logical" memory leak, caused by unintentionally retained references) rather than the classic C++ leak (forgetting to free memory that's already correctly known to be unreachable/unneeded) — Java shifts the failure mode rather than eliminating leaks as a concept entirely.

Q8. Scenario: A C++ class `Logger` maintains a private `vector<string>* entries` (a pointer to a heap-allocated vector), allocated in the constructor and freed in the destructor. A developer needs a free function, `dumpToFile(Logger& logger)`, that writes all of `Logger`'s log entries to a file, and wants to avoid adding a public getter for `entries` since it should remain fully encapsulated from ordinary calling code. How would `friend` help here, and what alternative should be considered first?

Answer: Declaring `dumpToFile` as a `friend` function of `Logger` would let it directly access the private `entries` pointer without requiring a public getter — this is exactly the kind of narrow, justified use case `friend` is meant for: one specific, trusted function needs direct internal access, without exposing that access broadly through the class's public interface. That said, the alternative worth considering first is simply adding a public method to `Logger` itself that provides exactly the needed capability at the right level of abstraction — e.g., a `Logger::writeEntriesTo(ostream&)` member method, which keeps the "how do I access my own entries" logic inside the class (where it naturally belongs) rather than granting an external function an unrestricted view into `Logger`'s internal representation. `friend` should generally be reached for only when this kind of clean public-interface solution genuinely isn't available or natural — using it as a default first choice for "I don't want to write a getter" is usually a sign the class's own interface is underdeveloped rather than a genuine need for the friend mechanism.